

Licensing Scheme Design Specification

Our specification should cover **all categories of licenses**, from fully open creative licenses to proprietary commercial or restricted use. This includes public-domain or Creative Commons (CC) styles (e.g. CC BY, CC BY-SA, etc.), as well as standard commercial stock licenses and “all rights reserved” cases. For example, content can be classified as **proprietary** (no reuse allowed, all rights reserved), **licensed** (use granted under conditions like payment or attribution), or **public domain** ¹ ². Ensuring these categories are supported means the format can express open reuse (with or without attribution), commercial use permissions, and restrictive copyright notices.

License Reference and Metadata Schema

- **Full License Links:** At minimum, each asset’s metadata should include a canonical URL or file pointer to the full license text. This follows best practices like the SPDX standard, which uses a permanent URL and short identifier for each license ³. Embedding a license *verification link* within an asset (for example in a file’s copyright field) is recommended ⁴. Such a link points back to a page or repository controlled by the owner (for verification and details) ⁴.
- **License Attributes:** In addition to the link, the schema should capture key license attributes as structured fields. For example: “allows reuse,” “allows modification/remix,” “allows commercial use,” “requires attribution,” etc. Creative Commons licenses are a useful model: they explicitly denote permissions like “reuse” and “remix” under conditions ⁵. Including these attributes in metadata ensures machines and users can quickly check if an asset can be repurposed, adapted, or used commercially without reading the full legal text.
- **Machine-Readable Metadata:** License information should be encoded in a standard way (e.g. RDF or JSON-LD metadata). For instance, Creative Commons provides RDF metadata templates for license info ⁶. Embedding such metadata in file headers or accompanying sidecar files makes the data machine-readable. This allows automated tools to discover that “this image is CC BY-SA 4.0” or “this audio is royalty-free” without manual inspection.

Asset and Concept Inclusion

Since this specification defines a design, it should allow **conceptual assets** and **physical files** to be included. The spec can include descriptions or concept art for characters (as intangible creative works), and attach actual asset files (graphics, models, audio) if available. Each asset entry would carry its own license metadata according to the schema. This approach lets users document not just the idea of a character or scene, but also the specific image/audio files, with clear licensing on each. (For example, an author might attach a concept sketch under CC BY, and a final image under a different license.) Explicitly permitting assets to be “attached” in the spec ensures that license information can be bound to each resource.

Multi-Asset Licensing

We support **bundles or groups of assets** under one specification. Each asset can have its own license, or a single license may cover multiple assets, depending on how they are packaged. For example, if a download includes several icons or 3D models, the user should be able to apply different licenses per asset or a unified license for the whole bundle. In practice, platforms like Envato require that if an item contains multiple assets, you license each end product separately ⁷; our spec can mirror that logic. We should allow metadata that either links each asset to its license or indicates a combined license for the set.

- **License Inheritance:** When assets are derived from or composed of other assets, our spec should define inheritance rules. A hierarchical approach (like NFTs or asset graphs) can let a child asset “inherit” license terms from its parent collection ⁸. For instance, if a character uses a background texture, and that texture is CC BY-SA, the character’s license might need to reflect that requirement. We must specify how licenses combine: e.g. requiring the strictest terms (copyleft) or using SPDX-style expressions (AND/OR) to represent multi-license scenarios ³.

Platform-Specific License Metadata

Different platforms or engines often have specialized license models (e.g. Unity Asset Store’s single-entity vs multi-entity seats ⁹). Our specification should allow **optional metadata blocks for platform-specific licenses**. For example, a user could supply an additional license section for “Unity Asset Store use” or “Web distribution,” if needed. These overrides would supplement the base license terms. We need clear rules: typically the platform-specific terms should not *weaken* the base license, only add constraints. For instance, if the base license allows reuse, a platform block could impose that each reuse requires a separate paid license (similar to Envato’s per-product licensing) ⁷.

- **Inheritance and Overrides:** We should document which license takes precedence. A sensible approach is: start from a default (most-permissive) license for an asset, then apply any platform overrides or included licenses. If conflicts arise, require that the most restrictive condition wins. This avoids legal issues where someone assumes broader rights than actually granted. For example, if the overall character spec is CC BY, but a specific image asset is CC BY-NC, the non-commercial restriction must apply for that image.

Granularity and Permissions

Granularity is crucial. We want the specification to clearly list **each character or asset element** and its permissions. Ideally, every resource (image, model, text) has its own license fields (permissions, author, attribution). This avoids ambiguity: downstream users can see “Character A (by Author X) is CC BY-SA” while “Background B (by Author Y) is CC BY-ND,” etc. By being explicit, we **help users avoid inadvertent copyright violations**.

In summary, the spec should support:

- **All license types** (open, commercial, proprietary) ¹ ²
- Pointers to full license texts (URLs or SPDX IDs) ³ ⁴
- Schema fields for reuse/remix/commercial permissions ⁵
- Machine-readable metadata for license info ⁶
- Multiple assets per bundle, with each asset’s license or combined license rules ⁷ ⁸

- Optional platform-specific license clauses or blocks
- Clear inheritance/override rules so that the strictest license applies in combinations.

By covering these points, the specification will clearly define who owns what rights and how content may be reused across platforms—minimizing legal confusion and maximizing adoption ⁵ ⁸ .

Sources: Authoritative discussions on licensing and metadata standards (e.g., Creative Commons and SPDX) ³ ⁴ ¹ ⁸ guide these recommendations. Each point above is derived from best practices for digital asset licensing and management.

¹ ² Understanding licenses – 99designs Help Center

<https://support.99designs.com/hc/en-us/articles/29111262151444-Understanding-licenses>

³ SPDX License List | Software Package Data Exchange (SPDX)

<https://spdx.org/licenses/>

⁴ Embedded Metadata - Creative Commons

https://wiki.creativecommons.org/wiki/Embedded_Metadata

⁵ Use & remix - Creative Commons

<https://creativecommons.org/share-your-work/use-remix/>

⁶ Metadata - Creative Commons

<https://wiki.creativecommons.org/wiki/metadata>

⁷ License Terms — Envato Elements

<https://elements.envato.com/license-terms>

⁸ Content ARCs: Decentralized Content Rights in the Age of Generative AI

<https://arxiv.org/html/2503.14519v1>

⁹ Asset Store licenses: Extension Assets, Single and Multi Entity assets – Unity Support Help Center

<https://support.unity.com/hc/en-us/articles/208601846-Asset-Store-licenses-Extension-Assets-Single-and-Multi-Entity-assets>